

Proposta

Proposta

Geração de testes orientados a TDD a partir de documentação de software utilizando LLMs

1. Contextualização

A documentação de software descreve o comportamento esperado de um sistema por meio de requisitos, histórias de usuário, critérios de aceitação, casos de uso ou especificações funcionais. Esses artefatos são fundamentais para o processo de desenvolvimento, pois indicam quais funcionalidades devem ser implementadas e quais regras de negócio precisam ser respeitadas.

No contexto de testes de software, uma dificuldade recorrente é transformar essa documentação em casos de testes claros, completos e úteis. Esse processo costuma ser manual, demorado e dependente da interpretação dos testadores e desenvolvedores.

Com o avanço dos modelos de linguagem de grande escala, os LLMs, surge a possibilidade de utilizar inteligência artificial para apoiar a geração de testes a partir da documentação. Neste projeto, a proposta é investigar como um LLM pode transformar requisitos de software em casos de teste de alto nível e, em uma segunda etapa, apoiar a geração de testes automatizados alinhados à prática de TDD.

2. Problema

Como utilizar modelos de linguagem para gerar casos de teste a partir de documentação de software de forma estruturada, rastreável e útil para um processo orientado a TDD?

O problema envolve entender como o modelo pode identificar, a partir da documentação:

- comportamentos esperados;
- regras de negócio;
- entradas e saídas;
- pré-condições;
- cenários positivos;
- cenários negativos;
- casos de borda;
- critérios de aceitação;
- possíveis testes automatizados.

3. Justificativa

A geração de testes com apoio de IA pode reduzir o esforço manual de criação de testes e auxiliar equipes a transformar requisitos em artefatos de validação. Essa

proposta é relevante porque conecta três elementos importantes da engenharia de software:

1. documentação de requisitos;
2. projeto de casos de teste;
3. desenvolvimento orientado a testes, ou TDD.

No TDD, os testes são definidos antes ou durante a implementação da funcionalidade. Dessa forma, usar a documentação como ponto de partida para gerar testes pode ajudar a transformar requisitos em especificações executáveis, aproximando o que foi documentado do que será validado no código.

Além disso, o uso de LLMs permite investigar questões atuais em teste de software, como qualidade dos testes gerados por IA, necessidade de revisão humana, risco de alucinação e comparação com testes produzidos por humanos.

4. Objetivo Geral

Propor e avaliar uma abordagem baseada em LLMs para gerar casos de teste orientados a TDD a partir de documentação de software.

5. Objetivos Específicos

- Selecionar documentos de requisitos ou histórias de usuário como entrada do processo.
- Criar prompts para geração de casos de teste de alto nível a partir da documentação.
- Investigar se uma etapa intermediária de seleção de técnicas de teste melhora a geração dos casos de teste.
- Gerar casos de teste em linguagem natural contendo cenário, pré-condições, passos e resultado esperado.
- Relacionar os testes gerados aos requisitos de origem.
- Comparar os testes gerados por LLM com testes produzidos por humanos.
- Avaliar os testes gerados usando métricas como similaridade semântica, precision, recall e F1.
- Opcionalmente, transformar casos de teste de alto nível em testes automatizados, seguindo a lógica de TDD.

6. Escopo do Projeto

O foco principal do projeto será a geração de casos de teste funcionais de alto nível a partir de documentação de software.

Esses casos de teste devem ser úteis para orientar o desenvolvimento em TDD, mas a implementação de testes automatizados pode ser tratada como uma etapa opcional ou extensão do trabalho.

O escopo principal será:

- entrada: documentação textual de requisitos ou histórias de usuário;
- processamento: uso de LLM com prompts estruturados;

- saída principal: casos de teste de alto nível;
- avaliação: comparação com testes humanos usando similaridade semântica e métricas quantitativas.

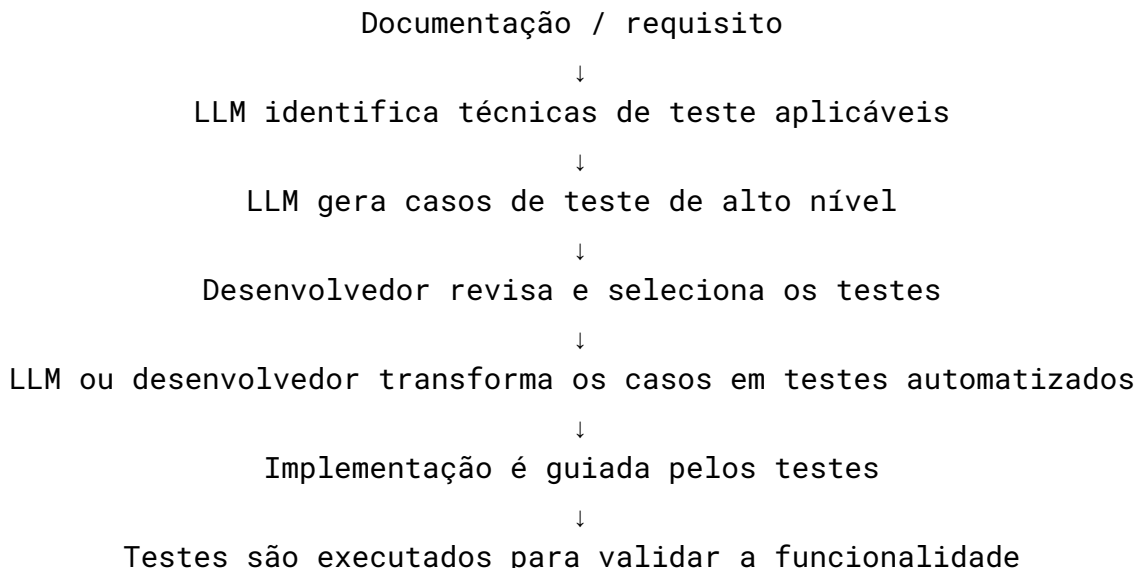
O escopo estendido poderá incluir:

- geração de código de teste a partir dos casos de teste de alto nível;
- uso de RAG para fornecer contexto adicional do sistema;
- avaliação dos testes automatizados por execução, cobertura ou mutation testing.

7. Relação com TDD

Neste projeto, TDD pode ser utilizado como uma perspectiva de organização dos testes. Em vez de gerar testes apenas depois da implementação, a abordagem proposta busca gerar testes a partir da documentação antes da codificação da funcionalidade.

O fluxo inspirado em TDD seria:



Dessa forma, a IA não substitui o processo de TDD, mas atua como apoio para transformar requisitos em cenários testáveis.

8. Questão de Pesquisa

Modelos de linguagem conseguem gerar casos de teste de alto nível, úteis para TDD, a partir de documentação de software com qualidade próxima à de testes produzidos por humanos?

8.1 Possíveis Perguntas Secundárias

- A geração em duas etapas, primeiro técnicas de teste e depois casos de teste, produz resultados melhores do que pedir diretamente os casos de teste ao LLM?
- Os testes gerados por IA cobrem os principais comportamentos descritos na documentação?

- Quais tipos de cenários são melhor ou pior gerados pelo modelo?
- Os testes gerados são úteis como ponto de partida para TDD?
- Quais limitações aparecem, como ambiguidade, testes redundantes ou cenários inventados?

9. Metodologia Proposta

9.1 Seleção da documentação base

Será escolhida uma documentação de software contendo requisitos ou histórias de usuário. Preferencialmente, a documentação deve possuir também testes ou casos de teste produzidos por humanos, para permitir comparação.

Possíveis fontes:

- documentação de sistemas open-source;
- requisitos públicos usados em artigos;
- histórias de usuário criadas pela equipe;
- documentação de um sistema pequeno desenvolvido em disciplina anterior.

9.2 Definição do benchmark humano

Será criado ou selecionado um conjunto de referência com casos de teste produzidos por humanos. Esse conjunto será usado como benchmark para avaliar os testes gerados pelo LLM.

Esse benchmark pode ser composto por:

- testes escritos manualmente pela equipe;
- casos de teste já existentes em projetos open-source;
- testes criados por outros estudantes, caso a professora permita;
- testes disponíveis junto com documentos de requisitos públicos.

9.3 Estratégias de geração com LLM

Serão comparadas duas estratégias principais.

Estratégia A: geração direta

O requisito é fornecido ao LLM, e o modelo é solicitado a gerar diretamente os casos de teste.

Requisito → LLM → Casos de teste

Exemplo de prompt:

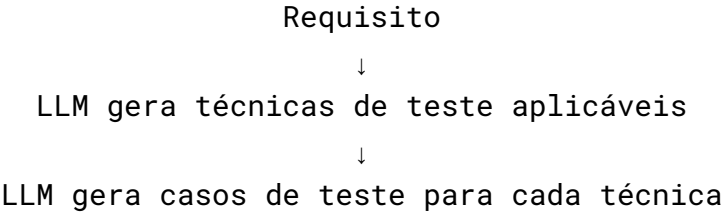
A partir do requisito abaixo, gere casos de teste funcionais de alto nível. Para cada caso, informe: ID, cenário, pré-condições, entradas, passos e resultado esperado.

Requisito:

[texto do requisito]

Estratégia B: geração em duas etapas

Primeiro, o LLM identifica técnicas de teste adequadas ao requisito. Depois, gera casos de teste com base nessas técnicas.



Exemplo de prompt:

Etapa 1:
Analisar o requisito abaixo e indique quais técnicas de teste funcional são mais adequadas.
Considere técnicas como particionamento de equivalência, análise de valor limite, testes positivos, negativos e casos de borda.

Etapa 2:
Com base nas técnicas identificadas, gere casos de teste de alto nível para o requisito.

Exemplos de técnicas que podem ser sugeridas:

- particionamento de equivalência;
- análise de valor limite;
- teste de fluxo principal;
- teste de fluxo alternativo;
- teste de campos obrigatórios;
- teste de regras de negócio;
- teste de estados;
- teste de casos negativos.

9.4 Estrutura dos casos de teste gerados

Cada caso de teste gerado deverá seguir um formato padronizado:

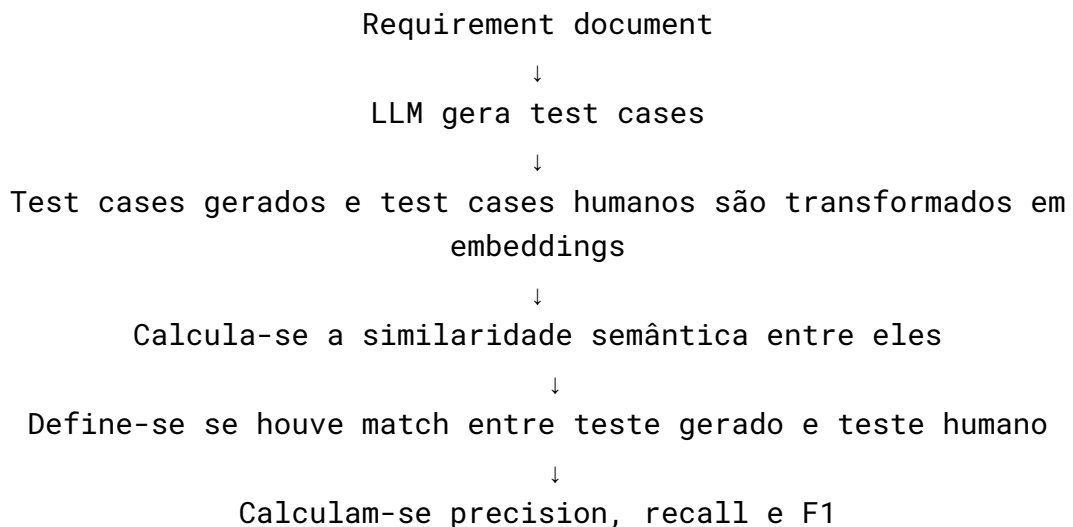
Campo	Descrição
ID	Identificador do caso de teste
Requisito relacionado	Requisito ou trecho da documentação usado como base
Técnica de teste	Técnica aplicada, quando houver
Cenário	Situação a ser testada
Pré-condições	Estado necessário antes da execução
Entradas	Dados usados no teste

Passos	Sequência de ações
Resultado esperado	Comportamento esperado do sistema
Tipo	Positivo, negativo ou caso de borda

9.5 Avaliação dos testes gerados

A avaliação será feita comparando os testes gerados pelo LLM com os testes humanos de referência.

Um possível fluxo de avaliação é:



As métricas podem ser interpretadas da seguinte forma:

- **Precision:** entre os testes gerados pelo LLM, quantos correspondem a testes relevantes do benchmark humano.
- **Recall:** entre os testes humanos esperados, quantos foram cobertos pelos testes gerados pelo LLM.
- **F1:** equilíbrio entre precision e recall.

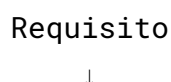
Além disso, pode ser feita uma análise qualitativa considerando:

- clareza dos testes;
- presença de alucinações;
- redundância;
- ausência de cenários importantes;
- utilidade para TDD;
- necessidade de revisão humana.

10. Possível extensão: geração de código de testes

Caso o escopo seja ampliado, os casos de teste de alto nível podem ser usados como entrada para gerar testes automatizados.

Exemplo de fluxo estendido:



Casos de teste de alto nível



Contexto do sistema via RAG



Geração de testes automatizados



Execução dos testes



Análise de falhas, cobertura ou mutation score

Essa etapa se aproxima mais diretamente de TDD, pois os testes automatizados poderiam ser escritos antes da implementação ou antes da correção de uma funcionalidade.

No entanto, essa extensão exige mais esforço, pois depende de código-fonte, ambiente de execução, framework de testes e definição clara dos oráculos de teste.

11. Trabalhos relacionados

A proposta se apoia em três linhas de trabalhos relacionados:

11.1 Avaliação de LLMs para teste de software

<https://www.sciencedirect.com/science/article/abs/pii/S0920548924001119>

Trabalhos recentes avaliam a capacidade de LLMs em gerar casos de teste, localizar bugs e apoiar atividades de teste. Uma ideia importante é comparar os resultados da IA com testes produzidos por humanos, usando estes como benchmark.

Essa abordagem é útil para o projeto porque permite avaliar os testes gerados de forma mais objetiva.

11.2 Geração de testes guiada por mutation testing

<https://dl.acm.org/doi/10.1145/3696630.3728544>

Outro trabalho relevante mostra o uso de LLMs combinados com mutation testing para gerar testes capazes de detectar falhas específicas, especialmente em cenários de privacidade, segurança e compliance.

Esse trabalho é mais avançado e industrial, então não será o foco principal do projeto. Porém, ele pode inspirar uma possível extensão futura: avaliar se os testes gerados realmente detectam falhas ou mutações no código.

11.3 Geração de casos de teste de alto nível a partir de requisitos

<https://arxiv.org/abs/2510.03641>

O trabalho mais próximo da proposta é o que gera casos de teste de alto nível diretamente a partir de documentos de requisitos usando LLMs. Uma contribuição interessante desse trabalho é não pedir diretamente os testes ao modelo, mas antes solicitar que ele identifique técnicas de teste adequadas para o requisito.

Essa decomposição pode melhorar a qualidade dos testes, pois força o modelo a estruturar melhor o raciocínio antes de gerar os cenários.

12. Entregáveis Esperados

Ao final do projeto, espera-se entregar:

- documento explicando a abordagem proposta;
- revisão breve dos trabalhos relacionados;
- conjunto de requisitos usados como entrada;
- conjunto de testes humanos usados como benchmark;
- prompts utilizados para geração de testes;
- casos de teste gerados pelo LLM;
- comparação entre geração direta e geração em duas etapas;
- avaliação quantitativa com precision, recall e F1;
- análise qualitativa dos resultados;
- discussão sobre uso da abordagem em TDD;
- Opcionalmente, protótipo simples para geração dos testes.

13. Resultados Esperados

Espera-se observar que LLMs conseguem gerar casos de teste úteis a partir da documentação, especialmente quando o requisito é claro e possui critérios de aceitação bem definidos.

Também se espera que a geração em duas etapas, com identificação prévia de técnicas de teste, produza casos mais organizados e com melhor cobertura do que a geração direta.

No entanto, é provável que os testes gerados ainda precisem de revisão humana, principalmente para evitar:

- testes inventados;
- interpretações incorretas;
- cenários redundantes;
- ausência de casos de borda;
- resultados esperados vagos;
- falta de aderência ao comportamento real do sistema.

14. Limitações

Algumas limitações previstas são:

- dificuldade de encontrar documentação pública com testes humanos correspondentes;
- subjetividade na comparação entre testes em linguagem natural;
- dependência da qualidade dos requisitos;
- risco de alucinação do LLM;
- dificuldade de avaliar automaticamente a utilidade real dos testes;
- necessidade de revisão humana;
- geração de código de teste pode exigir RAG, conhecimento da arquitetura e ambiente executável.

15. Cronograma

Etapa	Descrição
1	Definição do escopo: high-level test cases com foco em TDD
2	Seleção dos requisitos/documentação base
3	Criação ou seleção do benchmark humano
4	Definição dos prompts de geração direta
5	Definição dos prompts com técnicas de teste intermediárias
6	Geração dos casos de teste com LLM

7	Cálculo de similaridade semântica e métricas
8	Análise qualitativa dos testes gerados
9	Discussão sobre aplicação em TDD
10	Preparação da apresentação e relatório final

16. Conclusão da Proposta

Este projeto propõe investigar o uso de LLMs para gerar casos de teste a partir de documentação de software, com foco em apoiar práticas de TDD. A abordagem principal será transformar requisitos em casos de teste de alto nível, avaliando a qualidade desses testes por comparação com testes produzidos por humanos.

A proposta é viável para a disciplina porque permite trabalhar conceitos fundamentais de teste de software, como casos de teste, critérios de aceitação, técnicas de teste, oráculos, cobertura de requisitos e avaliação de qualidade. Ao mesmo tempo, incorpora um tema atual: o uso de inteligência artificial generativa no processo de teste.

Como extensão, o projeto poderá avançar da geração de casos de teste em linguagem natural para a geração de testes automatizados, possivelmente utilizando RAG para fornecer contexto do código-fonte e da arquitetura do sistema.

Metodologia

Desenho experimental

1. Objetivo do experimento

Avaliar se LLMs conseguem gerar casos de teste de alto nível a partir de documentação de software com qualidade semelhante a testes produzidos por humanos, e comparar se uma estratégia de geração em duas etapas produz melhores resultados do que uma estratégia de geração direta.

2. Questões de pesquisa

QP1: Os casos de teste gerados por LLM cobrem os principais comportamentos descritos nos requisitos?

QP2: A geração em duas etapas, primeiro técnicas de teste e depois casos de teste, apresenta melhores resultados do que a geração direta?

QP3: Os testes gerados são úteis como ponto de partida para uma abordagem TDD?

3. Variáveis do experimento

3.1 Variável independente

- estratégia de prompting;
- modelo utilizado, se forem testar mais de um;
- tipo de requisito.

A principal variável independente será a estratégia de geração: prompt direto versus prompt em duas etapas.

3.2 Variáveis dependentes

- precision;
- recall;
- F1;
- similaridade semântica;
- quantidade de testes gerados;
- quantidade de testes redundantes;
- quantidade de testes considerados incorretos;
- avaliação humana de clareza e utilidade.

4. Unidades experimentais

As unidades experimentais serão os requisitos de software selecionados. Para cada requisito, serão gerados casos de teste usando as duas estratégias de prompting.

5. Grupos comparados

Grupo	Descrição
Grupo A	Testes gerados por LLM com prompt direto
Grupo B	Testes gerados por LLM com prompt em duas etapas
Referência	Testes produzidos por humanos

6. Métricas

6.1 Precision

Entre os testes gerados pelo LLM, quantos têm correspondência com testes humanos.

$\text{Precision} = \text{matches encontrados} / \text{total de testes gerados pelo LLM}$

6.2 Recall

Entre os testes humanos esperados, quantos foram cobertos pelo LLM.

$\text{Recall} = \text{matches encontrados} / \text{total de testes humanos}$

6.3 F1

Média harmônica entre precision e recall.

$\text{F1} = 2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$

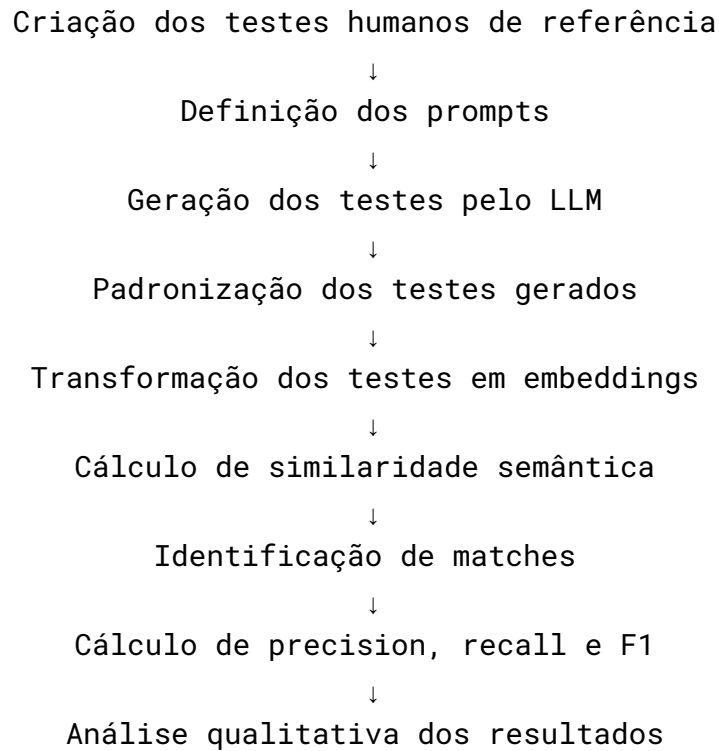
6.4 Métricas qualitativas:

Critério	Pergunta
Clareza	O teste é compreensível?
Executabilidade	O teste poderia ser executado por uma pessoa?
Rastreabilidade	O teste está ligado a um requisito?
Correção	O resultado esperado faz sentido?
Redundância	O teste repete outro cenário?
Alucinação	O teste inventa comportamento não documentado?
Utilidade para TDD	O teste ajuda a guiar a implementação?

7. Fluxo experimental

Seleção dos requisitos

↓



8. Ameaças à validade

Ameaça	Como mitigar
Benchmark humano pode estar incompleto	Mais de uma pessoa revisa os testes humanos
LLM pode gerar testes inventados	Marcar alucinações na análise qualitativa
Similaridade semântica pode errar matches	Revisão manual de casos duvidosos
Poucos requisitos podem limitar conclusões	Deixar claro que é estudo exploratório
Prompts podem favorecer uma estratégia	Usar prompts padronizados e documentados
Aleatoriedade do LLM	Fixar temperatura baixa e repetir execuções, se possível

9. Critérios de inclusão e exclusão dos requisitos

Incluir requisitos que:

- sejam funcionais;
- tenham comportamento verificável;
- tenham entradas e resultados esperados claros;

- possam gerar pelo menos um caso de teste.

Excluir requisitos que:

- sejam muito ambíguos;
- sejam puramente não funcionais;
- dependam de contexto externo não documentado;
- não permitam definir o resultado esperado.

Plano de coleta

1. Dados a serem coletados

Dado coletado	Descrição	Fonte
Requisitos	Textos usados como entrada	Documentação selecionada
Testes humanos	Casos de teste criados manualmente	Grupo/projeto de referência
Prompts	Prompts usados na geração	Elaborados pelo grupo
Respostas do LLM	Testes gerados automaticamente	Modelo de linguagem
Embeddings	Representações vetoriais dos testes	Modelo de embeddings
Similaridades	Scores entre testes humanos e gerados	Script de avaliação
Matches	Pares considerados equivalentes	Critério de similaridade
Métricas	Precision, recall e F1	Cálculo do grupo
Avaliação qualitativa	Clareza, utilidade, alucinação etc.	Revisão humana

2. Organização dos dados

dataset/

requirements/

req_001.txt

req_002.txt

human_tests/

req_001_tests.csv

req_002_tests.csv


```
llm_outputs/  
  direct_prompt/  
    req_001_output.csv  
  two_step_prompt/  
    req_001_output.csv  
  
prompts/  
  direct_prompt.txt  
  technique_prompt.txt  
  test_generation_prompt.txt  
  
evaluation/  
  semantic_similarity.csv  
  metrics_summary.csv  
  qualitative_analysis.csv
```

3. Formato da planilha principal

requirement_id	strategy	test_id	scenario	input	expected_result	type	technique
----------------	----------	---------	----------	-------	-----------------	------	-----------

- matches:

requirement_id	human_test_id	llm_test_id	similarity_score	match
----------------	---------------	-------------	------------------	-------

4. Critério de match

Será considerado match quando a similaridade semântica entre um teste gerado e um teste humano for maior ou igual a um limiar definido, por exemplo 0,75. Além disso, uma amostra dos matches será revisada manualmente para verificar se a correspondência faz sentido.

Uma alternativa mais robusta:

- similaridade $\geq 0,80$: match automático;
- entre 0,65 e 0,79: revisar manualmente;
- abaixo de 0,65: não match.

5. Quantidade de dados

O experimento inicial utilizará entre 10 e 20 requisitos. Para cada requisito, serão criados testes humanos de referência e gerados testes por duas estratégias de prompting. Espera-se coletar aproximadamente entre 100 e 200 casos de teste no total, considerando os testes humanos e os testes gerados pelo LLM.